

Journal de bord - TowerDefense

CHADEVILLE - TECHER

27 septembre 2025

1 Introduction

Dans le cadre de notre formation, module Computer Science, il nous est demandé de réaliser, en C++, une version simplifiée du Tower Defense, jeu emblématique des années 90. Le travail doit être mené en groupe de 2 ou 3 étudiants. Notre équipe est composée d'Alexandre TECHER et d'Enzo CHADEVILLE.

Ce document sera mis à jour régulièrement afin de suivre l'avancement du projet de manière bimensuelle. Il présentera nos idées, nos réussites, nos échecs ainsi que nos tentatives d'amélioration. Il fera office de journal de bord et constituera une trace du développement du projet.

2 Contraintes

Le projet comporte plusieurs contraintes imposées par notre enseignant :

- La carte de jeu doit contenir au moins un ou deux chemins possibles, ainsi que des zones ouvertes permettant le placement de tours
- Le terrain doit être conçu de manière à ce que le joueur puisse réfléchir à des stratégies visant à allonger le parcours des ennemis
- Si tous les chemins menant aux ressources sont bloqués, alors les ennemis ignorent les tourelles
- Le programme doit être développé en programmation orientée objet (Ennemis, Tours, Carte, ...).
- Le programme doit inclure un algorithme de « chemin le plus court »

3 Objectifs du projet

3.1 Mécanique de gameplay

Le jeu comporte une carte (map) sur laquelle le joueur pourra créer des tours afin d'éliminer les ennemis qui apparaissent.

La map, de forme rectangulaire et de dimensions (X, Y) , est divisée en trois zones :

- Une zone d'apparition des ennemis ;
- Une zone dans laquelle le joueur peut poser ses tours ;
- Une zone de fin de parcours pour les ennemis.

Un exemple de la disposition des zones sur la carte est présenté à la figure ??.



FIGURE 1 – Disposition des zones sur la carte.

Les ennemis apparaissent à une position aléatoire dans la zone de départ et doivent atteindre la fin du parcours en utilisant un algorithme de type "chemin le plus court".

Si un ennemi atteint la fin du parcours, le joueur perdra des points de vie.

Pour se défendre, le joueur peut placer des tours qui infligent des dégâts et peuvent également influencer le chemin des ennemis.

La figure ?? illustre l'impact du placement des tours sur la trajectoire des ennemis.

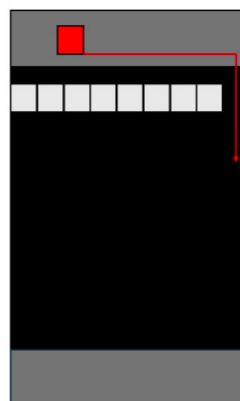


FIGURE 2 – Placement des tours pour dévier le chemin des ennemis.

Ainsi, plus le joueur place judicieusement ses tours, plus le chemin des ennemis s'allongera, augmentant les chances de les éliminer avant qu'ils n'atteignent la fin de la carte.

3.2 Inspiration

Pour notre Tower Defense, nous allons nous inspiré (au niveau du gameplay et une partie de l'HUD) de Tower Wars : StarCraft 2, un mode personnalisé créé dans l'éditeur de cartes de Starcraft 2.

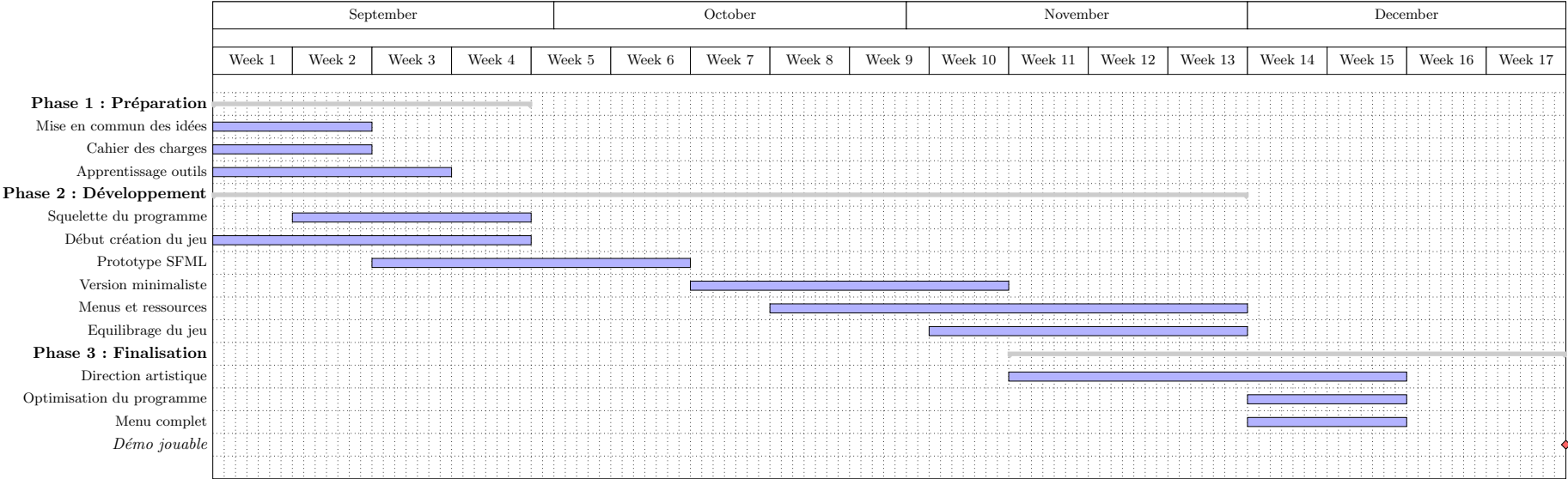
Nous nous inspirons également de Age Of empire pour son HUD des ressources.

4 Diagramme de classes



FIGURE 3 – Diagramme de classes du projet.

Planning prévisionnel (semaines 36 à 52)



5 Résumé d'activité des semaines 38-39

Durant ces deux semaines, nous avons entamé le développement des différents acteurs de notre projet Tower Defense. Nous avons profité de cette période pour approfondir nos connaissances en C++, notamment sur des notions essentielles telles que les pointeurs, les classes et l'héritage, les structures, les tableaux statiques et dynamiques, ainsi que la syntaxe générale. Nous nous sommes également intéressés à la manière de faire interagir nos différents acteurs (par exemple, les ennemis et les tourelles affichés dans une fenêtre SFML, ainsi que leurs comportements respectifs).

Afin de progresser efficacement, nous avons choisi de travailler chacun sur un programme distinct. Ce choix nous a semblé pertinent, car il permettait à chacun d'apprendre de manière autonome les bases indispensables au projet. Toutefois, cette répartition n'a pas empêché une réelle coopération : nous avons régulièrement partagé nos découvertes et nous nous sommes mutuellement aidés lorsque l'un d'entre nous rencontrait des difficultés.

Ce rapport propose un résumé de nos activités durant cette période. La première partie est consacrée aux travaux réalisés par Alexandre TECHER, tandis que la seconde présente ceux d'Enzo CHADEVILLE. Enfin, nous terminerons par une conclusion mettant en perspective ces deux semaines de travail et notre vision pour la suite du projet.

6 Alexandre TECHER

6.1 Classes

6.1.1 Classe Enemy

La classe Enemy représente les ennemis dans le jeu. Elle gère les attributs tels que la santé, la vitesse, et la position. Elle inclut également des méthodes pour déplacer les ennemis le long du chemin défini par l'algorithme de "chemin le plus court".

Elle contient une structure "enemy" qui regroupe les informations décrivant un ennemi (nom, forme (shape), position, vie, vitesse, carapace, attaque, vitesse d'attaque, portée, pathPoints, currentPathIndex, nombre d'ennemis).

"pathPoints" est un vecteur de `sf::Vector2f` qui contient les points du chemin que l'ennemi doit suivre. "currentPathIndex" est un entier qui indique l'index actuel dans le vecteur pathPoints.

Spawn Position() : Cette méthode initialise la position de l'ennemi sur la carte de manière aléatoire.

Move() : Cette méthode déplace l'ennemi le long du chemin défini par les points dans pathPoints. Elle utilise la vitesse de l'ennemi pour déterminer la distance à parcourir à chaque mise à jour. Grâce à la boucle "while", la fonction move() est exécutée en continue. Ainsi SolveAStar() est appelé régulièrement pour recalculer le chemin de l'ennemi en fonction de sa position actuelle.

Draw() : Cette méthode dessine l'ennemi à sa position actuelle sur la fenêtre de jeu.

Pour contenir les caractéristiques et le chemin de chaque ennemi, un vecteur "groupe_enemies" est utilisé. Chaque élément du vecteur est une instance de la structure "enemy".

6.2 Classe Tower

La classe Tower représente les tours que le joueur peut placer sur la carte. Elle gère les attributs tels que le type de tour, les dégâts, la portée, et le coût. Elle inclut des méthodes pour attaquer les ennemis lorsqu'ils entrent dans sa portée.

6.3 Classe Pathfinding AStar

Afin de permettre aux ennemis de se déplacer de manière intelligente sur la carte, un algorithme de pathfinding est nécessaire. Dans un premier temps, une lecture de la description de la librairie `boost_graph` permet de cerner les algorithmes de pathfinding disponibles.

lien de description : <https://www.boost.org/doc/libs/latest/libs/graph/doc/index.html>

L'algorithme Dijkstra's Shortest Paths a été envisagé. Mais après réflexion et comparaison sur wikipédia, l'algorithme A* (A-star) a été préféré car il est généralement plus efficace pour les jeux vidéo et les applications en temps réel.

La classe Pathfinding AStar implémente l'algorithme A* pour trouver le chemin le plus court entre deux points sur la carte. Elle est utilisée par la classe Enemy pour déterminer le chemin à suivre.

Tout d'abord, le programme de A* est inspiré du github suivant :
https://github.com/OneLoneCoder/Javidx9/blob/master/ConsoleGameEngine/SmallerProjects/OneLoneCoder_PathFinding_AStar.cpp

La vidéo suivante explique le fonctionnement de l'algorithme A* :

<https://www.youtube.com/watch?v=icZj67PTFhc>

Après modification du code source pour une utilisation avec SFML, l'algorithme A* fonctionne de la manière suivante :

- La carte est représentée comme une grille de nœuds, où chaque nœud correspond à une cellule de la carte. Chaque nœud a des attributs tels que son état (libre ou occupé), ses coordonnées, le coût pour atteindre ce nœud depuis le nœud de départ, le coût estimé pour atteindre le nœud d'arrivée depuis ce nœud, un vecteur contenant ses voisins, et un pointeur qui sera dirigé vers son nœud parent.
- CreateNodes() : Cette méthode initialise la grille de nœuds en fonction de la carte du jeu. Elle crée des nœuds pour chaque cellule de la carte et définit leurs attributs initiaux.
- EnemyNode() : Cette méthode traduit la position actuelle de l'ennemi en coordonnées de nœud dans la grille. Elle n'est pas forcément située au centre d'un nœud.
- SolveAStar() : C'est la méthode principale qui implémente l'algorithme A*. Elle prend en entrée les coordonnées de l'ennemi, et retourne un vecteur de nœuds représentant le chemin le plus court trouvé.
- Draw() : Cette méthode est utilisée pour visualiser la grille de nœuds et le chemin trouvé. Elle dessine chaque nœud avec une couleur différente en fonction de son état, et trace le chemin en reliant les nœuds du chemin trouvé.

Le résultat obtenu après l'exécution de l'algorithme A* est figuré ci-dessous :

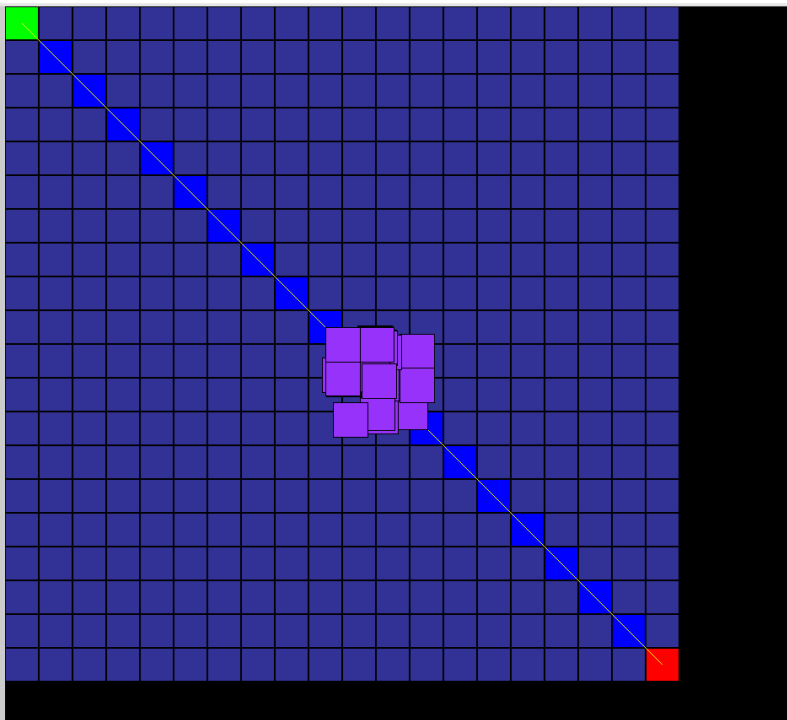


FIGURE 4 – Déplacement des ennemis avec l'algorithme A*

Le chemin le plus court est représenté par une ligne jaune reliant les nœuds verts (nœud de départ) et rouge (nœud d'arrivée). Les nœuds bleus clairs représentent les nœuds libres visités par l'algorithme, tandis que les nœuds bleus foncés représentent les nœuds libres non visités.

Ici, seulement un nœud est vert, car on affiche le nœud de départ du premier ennemi apparu.

6.4 Difficultés rencontrées

Les principales difficultés rencontrées à ce stade du projet ont été les suivantes :

La mise en place de l'algorithme A* pour le pathfinding des ennemis. Cela a nécessité une compréhension approfondie de l'algorithme et son adaptation à la librairie SFML.

l'échange de données entre les différentes classes, notamment entre la classe Enemy et la classe Pathfinding AStar.

la définition et la gestion du mouvement fluide des ennemis le long du chemin calculé par l'algorithme A*.

Important : Après plusieurs tentatives de résolution, ChatGPT a été utilisé pour résoudre certaines erreurs de compilation mais aussi pour surmonter les difficultés lors de la création de A* et du mouvement des ennemis.

6.5 Pistes d'amélioration

La structure des ennemis peut être remplacée par des classes dérivées pour chaque type d'ennemi. Cela permettrait une meilleure organisation du code et une plus grande flexibilité pour ajouter de nouveaux types d'ennemis avec des comportements spécifiques.

La création de tours devra être implémentée, avec des classes dérivées pour chaque type de tour. Chaque classe de tour pourrait avoir des attributs et des méthodes spécifiques, comme le type de dégâts infligés, la portée, et la vitesse d'attaque.

A* devra prendre en compte les tours placées par le joueur pour recalculer le chemin des ennemis en temps réel. Cela ajouterait une dimension stratégique au jeu, car les joueurs pourraient influencer le chemin des ennemis en plaçant des tours de manière stratégique.

De plus, au lieu de prendre en compte qu'un seul nœud de fin, A* pourrait être modifié pour prendre en compte plusieurs nœuds de fin, représentant plusieurs points d'arrivée possibles pour les ennemis.

Cela permettrait aux ennemis de choisir le chemin le plus court vers n'importe quel point d'arrivée.

7 Enzo CHADEVILLE

7.1 Les classes

Durant ces deux semaines, j'ai majoritairement travaillé sur l'apprentissage des classes et des classes héritières. La quasi-totalité de la semaine 38 a été utilisée pour comprendre concrètement comment ces classes peuvent communiquer. J'ai d'abord cherché à établir plusieurs types d'ennemis, définir arbitrairement leurs statistiques, réfléchir aux variables qui nous seraient utiles pour le jeu, déterminer ce que je devais mettre dans le constructeur, comment stocker mes ennemis, etc.

```
include > enemy.hpp > Enemy
1  #ifndef ENEMY_HPP
2  #define ENEMY_HPP
3
4  #include <SFML/Graphics.hpp>
5  #include <random>
6  #include <cstdlib> // pour rand() et srand()
7  #include <ctime> // pour time()
8
9
10
11 class Enemy // Déclaration de la classe Enemy
12 {
13     //private:
14
15     protected : //Protégé = accessible aux classes dérivées (les différents types d'ennemis)
16
17     //Paramètres de base de l'ennemi
18     int health;
19     float speed;
20     bool dead; // Variable d'état de l'ennemi (vivant/mort)
21     bool eat; // Variable d'objectif atteint
22
23
24     // Tableau de pointeurs vers des ennemis, comprenant la classe enemy, il porte le nom de "ennemies"
25     std::vector<Enemy*> enemies;
26     //Apparition des ennemis
27     int r = static_cast<int>(rand() % 5); // entre 0 et 3
28
29
30     //Définition de la forme de l'ennemi
31     sf::CircleShape shape;
32
33     //On définit à la construction de l'ennemi sa vie, vitesse, sa taille et sa couleur (pour l'instant c'est des forme ronde de couleur)
34     Enemy(int hp, float spd, float radius, sf::Color color); // Constructeur générique d'un ennemi, utile pour les type d'ennemis dérivés
```

FIGURE 5 – "enemy.hpp" – Classe Enemy, déclaration et partie `protected`

Sur l'ensemble de mes programmes, j'ai cherché à commenter la fonction de chaque ligne. Prendre rapidement cette habitude m'a permis de mieux me repérer dans mes fonctions, et de mieux comprendre le fonctionnement de certaines méthodes incluses dans les bibliothèques SFML ou encore les constructeurs. J'ai également « archivé » des parties de programme qui ne fonctionnaient pas ou qui ne me servaient plus. Sur la figure 1 se trouve la déclaration et le début de la classe « maîtresse ». Cette classe Enemy constituera la base de nos ennemis. On donne donc au constructeur certaines informations récurrentes nécessaires à tous les ennemis, à savoir leur vie, leur vitesse, leur taille et leur couleur pour l'affichage dans SFML. Nous pourrions implémenter au besoin plusieurs fonctions supplémentaires assez facilement.

```

include > enemy.hpp > Enemy
1  #ifndef ENEMY_HPP
11 class Enemy // Déclaration de la classe Enemy
35
36 public:
37 virtual ~Enemy() = default; // Destructeur virtuel par défaut -- apparemment obligatoire
38
39 static int counter; // Compteur d'ennemis total
40
41 //Gestion des points de vie
42 void hit(int amount); // L'ennemi subit des dégats
43 bool isDead() const; // L'ennemi est-il mort ?
44
45 //Etat de l'ennemi -- De simple getter pas forcément utile
46 int getHp() const;
47 float getSpeed() const;
48 bool hasEaten() const;
49
50 //--- SFML ---
51
52 void setPosition(float x, float y) { shape.setPosition(x, y); }
53
54 sf::Vector2f getPosition() const { return shape.getPosition(); }
55
56 // Boucle de jeu
57 virtual void update(float dt); // déplacement basique
58
59 virtual void draw(sf::RenderWindow& win) const // rendu
60 {
61     if (!isDead) win.draw(shape);
62 }
63 };
64
65

```

FIGURE 6 – "enemy.hpp" – Classe Enemy, partie public, fonctions généralisées

Dans la partie publique de la classe Enemy, je déclare les fonctions qui permettront de gérer les ennemis. J'ai également implémenté la fonction de compteur que nous avons vue en cours pour calculer le nombre d'ennemis. Pour l'instant, elle ne nous sert pas à grand-chose, puisque Alexandre et moi ne nous basons pas sur cette variable pour suivre nos ennemis. Ne m'étant pas encore occupé de la partie pathfinding, les ennemis n'ont pas de comportement complexe : seule leur forme et leur apparition sur une carte sont gérées.

Voici un exemple de quelques classes héritières de Enemy. Pour l'instant, je n'ai pas implémenté de fonctions propres à ces classes, mais on constate que, grâce à leur constructeur, je peux modifier manuellement les paramètres de la classe héritière. Ce système est vraiment intéressant, car il permet de créer différents types d'ennemis avec de simples variations de statistiques. Lorsque nous aurons suffisamment avancé, nous ajouterons des fonctions internes. Pour le moment, le fait de les différencier uniquement par leurs statistiques nous montre déjà que, grâce à une classe de base, nous pouvons générer une grande variété de déclinaisons.

J'ai pris la décision de déclarer les statistiques des ennemis directement dans le constructeur plutôt qu'au moment de leur génération. L'avantage est que la déclaration est centralisée dans un seul endroit et rend explicite le fait que les ennemis d'une classe donnée, comme FastEnemy, sont tous identiques.

```

1 //-----
2 // Ennemi tank (comportement identique aux ennemis de base,
3 // hp et vitesse changent)
4
5 class TankEnemy : public Enemy
6 {
7 public:
8     static int counter; // Compteur d'ennemis de type TankEnemy
9     TankEnemy() : Enemy(300, 1, 10.f, sf::Color(0,0,255)) { ++counter; }
10    ~TankEnemy() { --TankEnemy::counter; }
11 };
12
13 //-----
14 // Ennemi target (objectif : d truire les tourelles sur leur passage
15 // chemin le plus court en ignorant les tourelles, hp et vitesse changent)
16
17 class TargetEnemy : public Enemy
18 {
19 public:
20     static int counter; // Compteur d'ennemis de type TargetEnemy
21     TargetEnemy() : Enemy(150, 3, 4.f, sf::Color(100,100,100)) { ++counter;
22     }
23     ~TargetEnemy() { --TargetEnemy::counter; }
24 };

```

J'ai pu réutiliser ce squelette pour mes tours en changeant simplement les noms, il n'y a pas eu de difficulté notable sur la création des tours.

7.2 L'affichage

Pour l'instant l'affichage en SFML se fait sur deux fichiers, le `main.cpp` et le `render.cpp`. L'idée à terme est que toute la gestion du SFML soit gérée dans le fichier `render.cpp` afin de rendre le code plus propre, mais aussi de « nettoyer » le main pour n'y importer que les fonctions essentielles du genre `Game start`.

Ici le main gère l'initialisation du programme, c'est-à-dire la création de la fenêtre, l'initialisation des tableaux de pointeurs de type classe importante. On remarque le tableau de pointeurs de la classe `Enemy` : concrètement cette fonction permet de faire un tableau avec l'intégralité de nos ennemis, même si ce sont des classes héritières. Cela fonctionne parce qu'elles découlent toutes de la classe `Enemy`, ce qui est super pratique. Je pense qu'à terme nous fonctionnerons comme ça pour stocker nos ennemis, car il nous suffira de récupérer un identifiant de l'ennemi pour savoir à quelle espèce il appartient. On pourrait également faire plusieurs tableaux par espèce d'ennemis, mais si je dois me projeter je pense qu'en créer un par ennemi nous poserait des difficultés dans le cas où nous aurions besoin de faire une tourelle à AOE. En plus de cela, cela obligerait à effectuer toutes les actions de rendu et de traitement des ennemis globaux en N^* par type d'ennemi, ce qui n'est selon moi pas très pratique.

On fait également la même chose pour les tourelles puisque nous allons nous en servir de la même façon que pour les ennemis.

```
1 int main() {
2
3 //----- INITIALISATION -----//
4
5 // Initialisation de la fenetre SFML
6 sf::RenderWindow game_window(sf::VideoMode(800, 600), "Enemies_+SFML")
7 ;
8 game_window.setFramerateLimit(60);
9
10 // Initialisation des acteurs
11 std::vector<Enemy*> enemies; // PAS de new ici, pas besoin gr ce
12 // l instance de Game
13 std::vector<Tourelle*> tourelles; // PAS de new ici, pas besoin gr ce
14 // l instance de Game
15 std::vector<Render::Cell> cells; // Tableau de cellules pour la grille
16 cells = Render::buildCells(game_window); // On construit les cellules
17 // une fois pour toutes au d but
18 Game game;
19
20 // Initialisation des al toires
21 std::srand(static_cast<unsigned>(std::time(nullptr))); // seed une fois
22
23 // Initialisation d une clock
24 sf::Clock clock;
25 const float spawnoffset = 0.5f; // spawn toutes les 0.5s
```

Pour pouvoir gérer le jeu efficacement, j'ai pris la décision de faire dessiner une grille sur la fenêtre. Je vais me servir de cette grille pour faire apparaître des ennemis et des tourelles sur certaines cases. L'avantage de ça, c'est que ça rend selon moi plus pratique la pose de tourelle : plutôt que de dire « Je pose une tourelle au pixel (x,y) » et risquer de pouvoir poser une tourelle juste à côté au pixel voisin, ici on définit directement la taille d'une case. Pour faire une analogie, ce serait comme lorsque l'on place une structure dans *Clash of Clans* (ou n'importe quel jeu de gestion à grille comme *Age of Empires*), ou même *Minecraft* pour les jeux de survie.

```
include > render.hpp > ...
1  #ifndef RENDER_HPP
4  #include <SFML/Graphics.hpp>
6  #include <SFML/Graphics.hpp>
7
8  #include "enemy.hpp"
9
10 class Render {
11
12 public:
13
14     // ----- représentation d'une cellule ----- //
15     struct Cell {
16         int id; // 1-based (1,2,3,...) gauche->droite, haut->bas
17         int col, row; // indices 0-based
18         sf::FloatRect bounds; // rectangle en pixels (x,y,w,h)
19         sf::Vector2f center; // centre en pixels
20         bool turreted = false; // indique si une tourelle est placée dans cette cellule
21         static float cellSize;
22     };
23
24     // dessine une grille par-dessus la fenêtre, cellSize : taille d'une cellule en pixels
25     static void drawGrid(sf::RenderTarget& window, sf::Color color = sf::Color(60, 60, 60));
26
27
28
29     // Construit toutes les cellules visibles pour la fenêtre courante
30     static std::vector<Cell> buildCells(sf::RenderTarget& window);
31     // ----- //
32
33 };
34
```

FIGURE 7 – "render.hpp" – Classe Render, partie public, fonctions de la grille

Cependant, je ne pouvais pas (par souci de structure du programme) faire à la fois la fonction de dessin et la génération de mon tableau de cellules (j'appelle cellule une case dans la grille). J'ai donc dû les séparer.

La fonction « buildCells » est donc la fonction « pratique », c'est elle qui possède toutes les caractéristiques de ma cellule. Ma cellule comporte les caractéristiques suivantes :

```

4
5 // ----- VARIABLES PAR DEFALT ----- //
6 float Render::Cell::cellSize = 40.f; // Définition de la taille de cellule par défaut
7 float cellSize = Render::Cell::cellSize; // Variable globale pour la taille des cellules (utile dans main.cpp)
8 // ----- //
9
10
11 void Render::drawGrid(sf::RenderTarget& window, sf::Color color)
12 {
13
14     // --- Gestion de la grille pour SFML --- //
15     sf::Vector2u size = window.getSize(); // taille de la fenêtre (par défaut 800x600 (voir main.cpp))
16     float W = static_cast<float>(size.x); // largeur
17     float H = static_cast<float>(size.y); // hauteur
18
19     sf::VertexArray grid(sf::Lines); // dit qu'on va dessiner des lignes entre chaque point
20
21     // Lignes verticales
22     for (float x = 0.f; x <= W; x += cellSize) {
23         grid.append(sf::Vertex(sf::Vector2f(std::round(x), 0.f), color));
24         grid.append(sf::Vertex(sf::Vector2f(std::round(x), H), color));
25     }
26
27     // Lignes horizontales
28     for (float y = 0.f; y <= H; y += cellSize) {
29         grid.append(sf::Vertex(sf::Vector2f(0.f, std::round(y)), color));
30         grid.append(sf::Vertex(sf::Vector2f(W, std::round(y)), color));
31     }
32
33     window.draw(grid); // une fois qu'on a tout ajouté on dessine
34 }
35
36

```

FIGURE 8 – "render.hpp" – Classe Render, fonctions, drawGrid

```

38 std::vector<Render::Cell> Render::buildCells(sf::RenderTarget& window)
39 {
40     // --- Gestion de la grille pour le reste du programme --- //
41
42     sf::Vector2u size = window.getSize(); // taille de la fenêtre (par défaut 800x600 (voir main.cpp))
43     const int cols = static_cast<int>(size.x / cellSize); // permet dans le for de dire quel colonne/ligne on est
44     const int rows = static_cast<int>(size.y / cellSize);
45
46     std::vector<Cell> cells;
47     cells.reserve(cols * rows); //Eparqunement de mémoire pour éviter les reallocations (optimisation)
48
49     int id = 1; // On initialise l'id à 1 pour avoir un tableau qui dit cells 1 à cells n (on pourrais mettre 0)
50
51     for (int row = 0; row < rows; ++row) { // On a défini que nos incrémentation (rows, cols) était de la taille d'une cellule plutôt qu'un pixel
52         for (int col = 0; col < cols; ++col, ++id) { // On incrémente l'id à chaque cellule
53             const float x = col * cellSize;
54             const float y = row * cellSize;
55
56             // On implémente les paramètres de la nouvelle cellule dans notre structure Cell
57             Cell c;
58             c.id = id;
59             c.col = col;
60             c.row = row;
61
62             c.bounds = sf::FloatRect(x, y, cellSize, cellSize); // On définit le rectangle de la cellule qui part de (x,y) et fait cellSize en largeur et hauteur
63             c.center = { x + (cellSize/2), y + (cellSize/2) }; // On définit le centre de la cellule au milieu du rectangle
64             // On ajoute la cellule au tableau
65             cells.push_back(c);
66         }
67     }
68
69     return cells; // On retourne le tableau de cellules
70 }

```

FIGURE 9 – "render.cpp" – Classe Render, fonctions, buildCells

On a donc l'id qui permet d'identifier chaque cellule, row et col qui servent au calcul de leurs coordonnées, center qui permet d'indiquer les coordonnées x,y du centre de celle-ci, un booléen turreted qui permet de voir si une cellule a déjà une tourelle installée dessus (avantage : évite les empilements sur une seule cellule et rend l'affichage plus propre). Enfin, une variable « static » cellSize indique que chaque instance de cette structure possède la même taille, ce qui est pratique pour standardiser un processus. Enfin, les deux fonctions : « drawGrid », qui s'occupe du visuel, et « buildCells » pour le côté pratique.

Les deux fonctions font un balayage de la fenêtre : simplement, l'une dessine tandis que l'autre paramètre les cellules.

On constate que la fonction `drawGrid` nous permet d'obtenir ce rendu dans la fenêtre SFML :

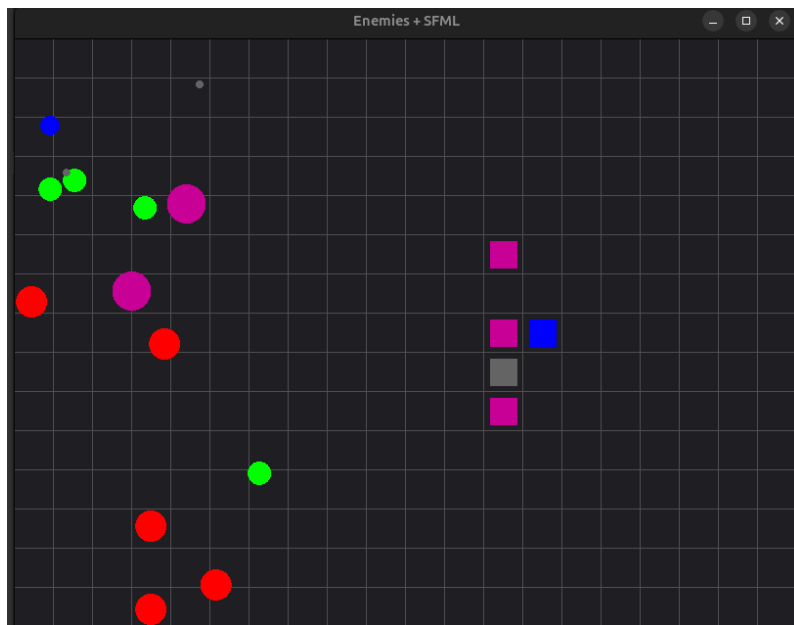


FIGURE 10 – Rendu SFML avec les grilles, ennemis (ronds), tourelles (carrés)

7.3 Gestion du jeu

Pour l’instant, j’ai créé les fonctions de génération d’ennemis et de tourelles. Les deux fonctions sont sensiblement identiques, à une différence près : dans ma fonction de génération de tourelles je prends en compte ce qui a été établi plus tôt avec les cellules.

D’un clic de souris, je décide de poser une tourelle : le programme récupère les coordonnées de ma souris puis les compare avec celles d’une cellule. C’est cette fonction qui m’a posé le plus de problèmes : j’avais du mal à saisir quand on pouvait modifier une structure et quand on ne le pouvait pas. Je ne passais pas par référence pour mon tableau de cellules : lorsque je voulais poser une nouvelle tourelle sur une case déjà utilisée, le programme crashait ou n’appliquait pas mes changements. En passant par référence, j’ai pu corriger les problèmes et la fonction a fini par marcher.

Je crée donc une sorte de « pointeur » nommé `it` qui agit sur la cellule correspondant à mon clic de souris. Référencer `cells` et `it` me permet de modifier les paramètres de la structure de cette cellule (par exemple la cellule avec l’id 27, donc la 27e cellule). Je peux ensuite établir deux conditions : - si ma cellule (id 27) possède déjà une tourelle (`turreted == true`), alors je ne fais rien ; - sinon, j’en génère une aléatoirement (pour le moment, c’est du test).

```
28 // ----- FONCTIONS DES TOURELLES ----- //
29
30
31 // Quand on a une structure ou un tableau de structure en paramètre, on doit passer par référence (&) pour pouvoir modifier les données
32 // Si on ne passe pas par référence, on travaille sur une copie locale de la structure,
33 // et les modifications ne sont pas visibles en dehors de la fonction
34 // Par contre si on passe par référence, on travaille directement sur la structure d'origine
35 // et les modifications sont visibles en dehors de la fonction
36 // Ici on passe par référence car on veut modifier le booléen turreted de la cellule
37 // Sinon ça marche juste pas on peut effectivement récupérer l'id de la cellule mais pas modifier son état (turreted = true) (pas le but)
38 // DONC PITIE RAPPEL TOI DU & POUR LES STRUCTURES/CLASSES/TABLEAUX STP
39
40 void Game::generateTourelle(std::vector<Tourelle*>& tourelles, std::vector<Render::Cell>& cells, float x, float y) {
41
42     if (cells.empty()) return; // pas de cellule, on ne peut rien faire
43
44     // Trouver la cellule qui contient le clic (x,y)
45     auto it = std::find_if(cells.begin(), cells.end(), [x,y]( Render::Cell& c){ return c.bounds.contains(x, y); });
46
47     if (it == cells.end()) {
48         // clic hors grille
49         return;
50     }
51
52     // Génération de la fonction rand (pour l'instant, apres ça sera avec les touches)
53     int r = std::rand() % 5; // 0..4
54
55     // On fait ça pour éviter de surcharger la lecture quand on crée des tourelles
56     const float cs = Render::Cell::cellSize;
57
58
59     if (it->turreted) {
60         // déjà une tourelle sur cette cellule
61         std::cout << "Cellule " << it->id << " déjà occupée !\n";
62         return;
63     }
64
65     // Pas de tourelle sur cette cellule alors banco piou piou
66     if (it->turreted == false) {
67         // la cellule est libre
68         switch (r) {
69             case 0: tourelles.push_back(new BasicTourelle((cs))); break;
70             case 1: tourelles.push_back(new PoisonTourelle((cs/2))); break;
71             case 2: tourelles.push_back(new shotgunTourelle((cs/2))); break;
72             case 3: tourelles.push_back(new TargetTourelle((cs/2))); break;
73             case 4: tourelles.push_back(new FlyTourelle((cs/2))); break;
74         }
75         it->turreted = true; // on marque la cellule comme occupée
76         std::cout << "Tourelle placée en cellule " << it->id << "\n";
77     }
78
79
80     // Positionne l'ennemi à la position de la souris
81     // Trouver la cellule la plus proche de la position de la souris
82     tourelles.back()->setPosition(it->center.x, it->center.y);
83
84 }
```

FIGURE 11 – “game.cpp” – Classe Game, fonctions, generateTourelle

7.4 Progression personnelle

Les différentes fonctions de mon programme (que vous pouvez retrouver sur GitHub dans la branche `test`) m'ont permis de mieux comprendre le fonctionnement du langage C++. Cette première ébauche d'un jeu de Tower Defense m'aura permis de poser les bases pour créer un jeu avec un code source propre, professionnel et exploitant les outils d'optimisation de C++. J'ai utilisé plusieurs outils pour m'aider dans mon apprentissage, notamment ChatGPT pour résoudre certaines erreurs de compilation et comprendre des concepts complexes (Je pense avant tout aux classes et leurs gestion). Mais également la documentation officielle de SFML (<https://www.sfml-dev.org/documentation/2.5.1/>) pour comprendre les différentes fonctions et classes disponibles dans cette bibliothèque graphique. (exemple : la gestion des formes géométriques, des fenêtres, des événements, etc.) et enfin l'outil de développement Copilot de GitHub pour m'aider à écrire du code et gagner du temps.

8 Conclusion des semaines 38-39

Ces deux semaines ont été très formatrices pour nous deux. Nous avons pu approfondir nos connaissances en C++ et en SFML, tout en commençant à développer les bases de notre jeu Tower Defense. Nous avons également appris à travailler de manière autonome tout en collaborant efficacement. Notre prochain objectif est d'intégrer nos travaux respectifs pour créer un prototype jouable, en continuant à développer les fonctionnalités du jeu et en améliorant l'interaction entre les différentes classes. Si on se réfère au planning prévisionnel, nous sommes encore dans les temps pour finaliser le projet dans les délais impartis.